Assignment 8: Time Series Analysis

Kyle Falk

Fall 2025

OVERVIEW

This exercise accompanies the lessons in Environmental Data Analytics on generalized linear models.

Directions

1. Rename this file `<FirstLast>_A08_TimeSeries.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change "Student Name" on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.

Set up

1. Set up your session:

- Check your working directory
- Load the tidyverse, lubridate, zoo, and trend packages
- Set your ggplot theme

```
library(here)
```

```
## here() starts at /home/guest/ENV 872/EDE_Fall2025
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   4.0.0     v tibble    3.2.1
## v lubridate 1.9.3     v tidyr     1.3.1
## v purrr     1.0.2

## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(lubridate)
library(zoo)
```

```
##
## Attaching package: 'zoo'
##
## The following objects are masked from 'package:base':
##
##     as.Date, as.Date.numeric
```

```r
library(trend)

here()
```

```
## [1] "/home/guest/ENV 872/EDE_Fall2025"
```

```r
my_theme <- theme_linedraw(base_size = 12) +
  theme(axis.text = element_text(color = "firebrick"),
        legend.position = "right")
theme_set(my_theme)
```

2. Import the ten datasets from the Ozone_TimeSeries folder in the Raw data folder. These contain ozone concentrations at Garinger High School in North Carolina from 2010-2019 (the EPA air database only allows downloads for one year at a time). Import these either individually or in bulk and then combine them into a single dataframe named `GaringerOzone` of 3589 observation and 20 variables.

```r
#1
Ozone_2010 <- read.csv(here("Data/Raw/Ozone_TimeSeries/EPAair_O3_GaringerNC2010_raw.csv"),
                       stringsAsFactors = TRUE)
Ozone_2011 <- read.csv(here("Data/Raw/Ozone_TimeSeries/EPAair_O3_GaringerNC2011_raw.csv"),
                       stringsAsFactors = TRUE)
Ozone_2012 <- read.csv(here("Data/Raw/Ozone_TimeSeries/EPAair_O3_GaringerNC2012_raw.csv"),
                       stringsAsFactors = TRUE)
Ozone_2013 <- read.csv(here("Data/Raw/Ozone_TimeSeries/EPAair_O3_GaringerNC2013_raw.csv"),
                       stringsAsFactors = TRUE)
Ozone_2014 <- read.csv(here("Data/Raw/Ozone_TimeSeries/EPAair_O3_GaringerNC2014_raw.csv"),
                       stringsAsFactors = TRUE)
Ozone_2015 <- read.csv(here("Data/Raw/Ozone_TimeSeries/EPAair_O3_GaringerNC2015_raw.csv"),
                       stringsAsFactors = TRUE)
Ozone_2016 <- read.csv(here("Data/Raw/Ozone_TimeSeries/EPAair_O3_GaringerNC2016_raw.csv"),
                       stringsAsFactors = TRUE)
Ozone_2017 <- read.csv(here("Data/Raw/Ozone_TimeSeries/EPAair_O3_GaringerNC2017_raw.csv"),
                       stringsAsFactors = TRUE)
Ozone_2018 <- read.csv(here("Data/Raw/Ozone_TimeSeries/EPAair_O3_GaringerNC2018_raw.csv"),
                       stringsAsFactors = TRUE)
Ozone_2019 <- read.csv(here("Data/Raw/Ozone_TimeSeries/EPAair_O3_GaringerNC2019_raw.csv"),
                       stringsAsFactors = TRUE)

GaringerOzone <- full_join(Ozone_2010, Ozone_2011) %>%
  full_join(Ozone_2012) %>%
  full_join(Ozone_2013) %>%
```

```
  full_join(Ozone_2014) %>%
  full_join(Ozone_2015) %>%
  full_join(Ozone_2016) %>%
  full_join(Ozone_2017) %>%
  full_join(Ozone_2018) %>%
  full_join(Ozone_2019)
```

Wrangle

3. Set your date column as a date class.

4. Wrangle your dataset so that it only contains the columns Date, Daily.Max.8.hour.Ozone.Concentration, and DAILY_AQI_VALUE.

5. Notice there are a few days in each year that are missing ozone concentrations. We want to generate a daily dataset, so we will need to fill in any missing days with NA. Create a new data frame that contains a sequence of dates from 2010-01-01 to 2019-12-31 (hint: `as.data.frame(seq())`). Call this new data frame Days. Rename the column name in Days to "Date".

6. Use a `left_join` to combine the data frames. Specify the correct order of data frames within this function so that the final dimensions are 3652 rows and 3 columns. Call your combined data frame GaringerOzone.

```
# 3
GaringerOzone$Date <- mdy(GaringerOzone$Date)

# 4
GaringerOzoneProcessed <- GaringerOzone %>%
  select(Date, Daily.Max.8.hour.Ozone.Concentration, DAILY_AQI_VALUE)

# 5
Days <- as.data.frame(seq(as.Date('2010-01-01'), as.Date('2019-12-31'), by = 'days'))
colnames(Days) <- "Date"
# 6
GaringerOzone <- left_join(Days, GaringerOzoneProcessed)
```

```
## Joining with `by = join_by(Date)`
```

Visualize

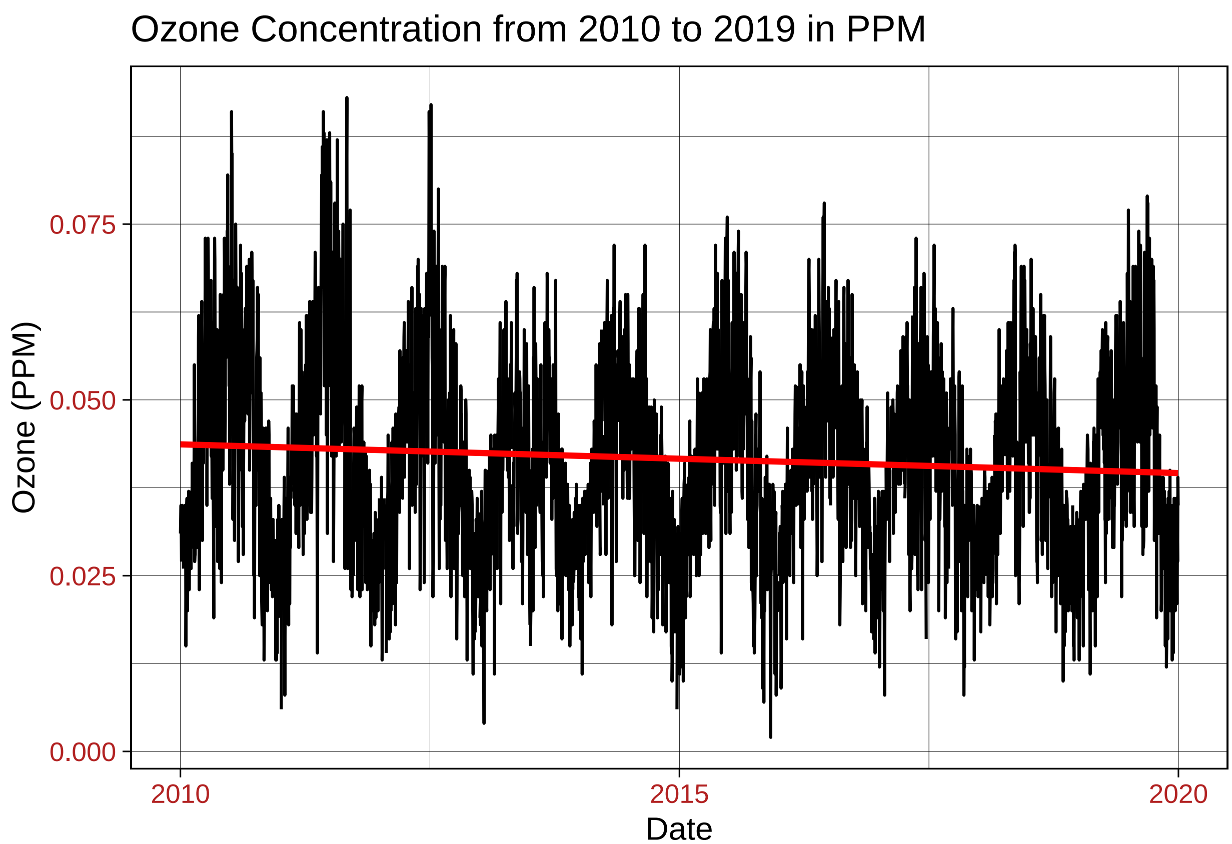
7. Create a line plot depicting ozone concentrations over time. In this case, we will plot actual concentrations in ppm, not AQI values. Format your axes accordingly. Add a smoothed line showing any linear trend of your data. Does your plot suggest a trend in ozone concentration over time?

```
#7
Ozone_over_time <- GaringerOzone %>%
  ggplot(
    aes(x=Date,
        y=Daily.Max.8.hour.Ozone.Concentration)) +
      geom_line() +
  geom_smooth(method = lm,
              se=FALSE,
```

```
                color="red") +
  labs(y="Ozone (PPM)",
       title = "Ozone Concentration from 2010 to 2019 in PPM")
print(Ozone_over_time)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

```
## Warning: Removed 63 rows containing non-finite outside the scale range
## (`stat_smooth()`).
```



Answer: The plot shows an overall seasonality to the data, with the ppm increasing in some parts of the year, and decreasing in others. There is regular spacing to the seasonality, with higher values being found in the summer. Additionally, there is a very slight decrease over time in the linear trend line, but not enough to meaningfully say that ozone is trending down over time.

Time Series Analysis

Study question: Have ozone concentrations changed over the 2010s at this station?

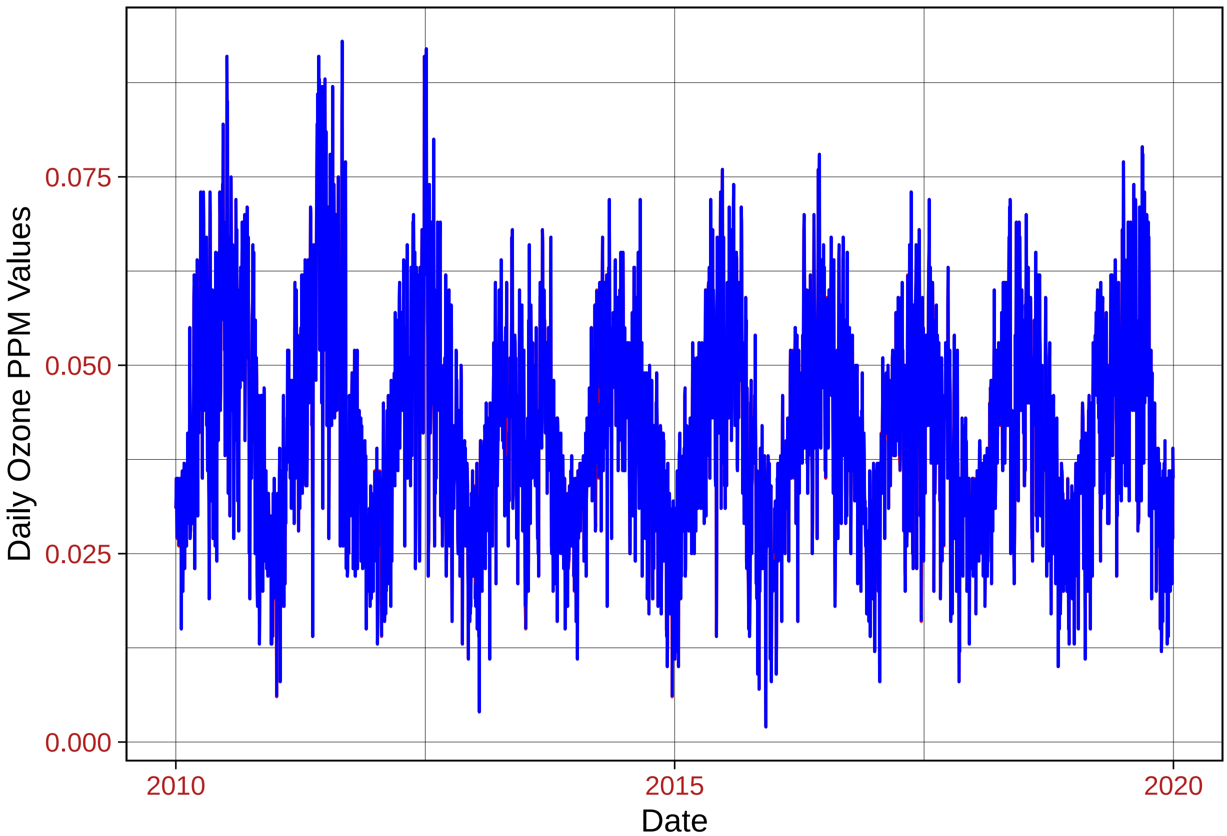
8. Use a linear interpolation to fill in missing daily data for ozone concentration. Why didn't we use a piecewise constant or spline interpolation?

```r
#8
GaringerOzoneInterpolation <- GaringerOzone %>%
  mutate(DailyPPM.Clean =zoo::na.approx(Daily.Max.8.hour.Ozone.Concentration))

summary(GaringerOzoneInterpolation)
```

```
##       Date            Daily.Max.8.hour.Ozone.Concentration DAILY_AQI_VALUE
##  Min.   :2010-01-01   Min.   :0.00200                      Min.   :  2.00
##  1st Qu.:2012-07-01   1st Qu.:0.03200                      1st Qu.: 30.00
##  Median :2014-12-31   Median :0.04100                      Median : 38.00
##  Mean   :2014-12-31   Mean   :0.04163                      Mean   : 41.57
##  3rd Qu.:2017-07-01   3rd Qu.:0.05100                      3rd Qu.: 47.00
##  Max.   :2019-12-31   Max.   :0.09300                      Max.   :169.00
##                       NA's   :63                           NA's   :63
##  DailyPPM.Clean
##  Min.   :0.00200
##  1st Qu.:0.03200
##  Median :0.04100
##  Mean   :0.04151
##  3rd Qu.:0.05100
##  Max.   :0.09300
##
```

```r
ggplot(GaringerOzoneInterpolation) +
  geom_line(aes(x = Date, y = DailyPPM.Clean), color = "red") +
  geom_line(aes(x = Date, y = Daily.Max.8.hour.Ozone.Concentration), color = "blue") +
  ylab("Daily Ozone PPM Values")
```



Answer: Interpolation was used because the missing data was only for short periods of time. We did not use a piecewise constant interpolation because that requires knowledge that the data is related and constant. Additionally, due to the seasonal nature, with periods of increasing and decreasing values, we do not want to assign an unknown day a value that is equivalent to the day before. We did not use spline interpolation because it is more useful for more complex data, with varying patterns. This is not something seen in our data, so it is not necessary to do a spline interpolation.

9. Create a new data frame called `GaringerOzone.monthly` that contains aggregated data: mean ozone concentrations for each month. In your pipe, you will need to first add columns for year and month to form the groupings. In a separate line of code, create a new Date column with each month-year combination being set as the first day of the month (this is for graphing purposes only)

```
#9
GaringerOzone.monthly <- GaringerOzoneInterpolation %>%
  mutate(GaringerOzoneInterpolation, Month = month(Date)) %>%
  mutate(GaringerOzoneInterpolation, Year = year(Date))

Monthly_Averages <- GaringerOzone.monthly %>%
  group_by(Year, Month) %>%
  summarize(Monthly_AveragePPM = mean(DailyPPM.Clean))
```

```
## `summarise()` has grouped output by 'Year'. You can override using the
## `.groups` argument.
```

```r
Months <- as.data.frame(seq.Date(from=as.Date("2010/01/01"),
                                 to=as.Date("2019/12/31"),
                                 by="month"))
colnames(Months) <- "Date"

GaringerOzone.monthly <- left_join(Months, GaringerOzone.monthly) %>%
  left_join(Monthly_Averages)
```

```
## Joining with `by = join_by(Date)`
## Joining with `by = join_by(Month, Year)`
```

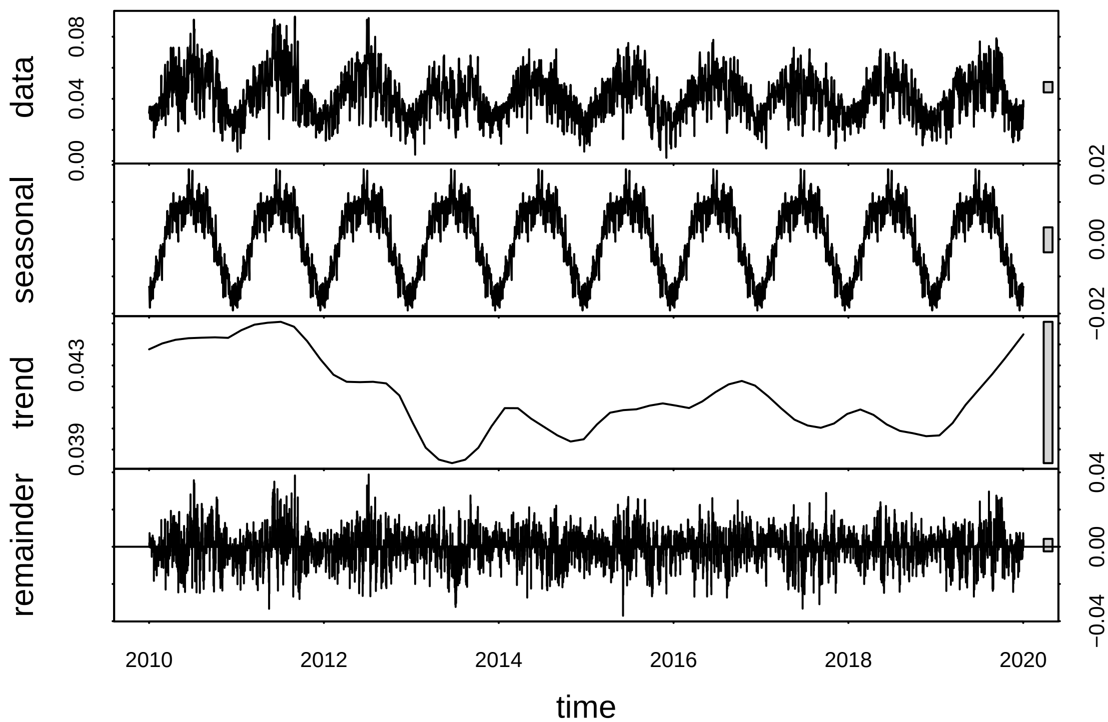
10. Generate two time series objects. Name the first `GaringerOzone.daily.ts` and base it on the dataframe of daily observations. Name the second `GaringerOzone.monthly.ts` and base it on the monthly average ozone values. Be sure that each specifies the correct start and end dates and the frequency of the time series.

```r
#10
GaringerOzone.daily.ts <- ts(GaringerOzoneInterpolation$DailyPPM.Clean,
                             start = c(2010,1,1),
                             frequency = 365)
GaringerOzone.monthly.ts <- ts(GaringerOzone.monthly$Monthly_AveragePPM,
                               start = c(2010,1,1),
                               frequency = 12)
```

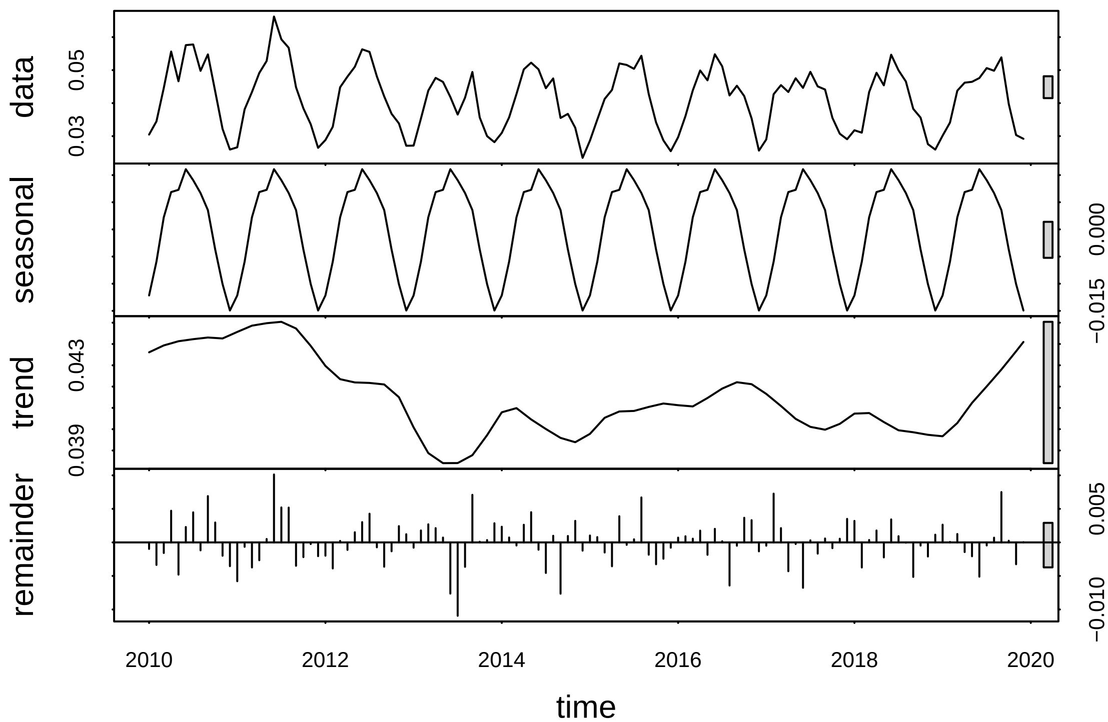
11. Decompose the daily and the monthly time series objects and plot the components using the `plot()` function.

```r
#11
GaringerOzone.daily.decomposed <- stl(GaringerOzone.daily.ts, s.window = "periodic")
GaringerOzone.monthly.decomposed <- stl(GaringerOzone.monthly.ts, s.window = "periodic")

plot(GaringerOzone.daily.decomposed)
```




```r
plot(GaringerOzone.monthly.decomposed)
```



12. Run a monotonic trend analysis for the monthly Ozone series. In this case the seasonal Mann-Kendall is most appropriate; why is this?

```
#12
MonthlyMannKendall <- Kendall::SeasonalMannKendall(GaringerOzone.monthly.ts)

MonthlyMannKendall
```

```
## tau = -0.143, 2-sided pvalue =0.046724
```

```
summary(MonthlyMannKendall)
```

```
## Score =  -77 , Var(Score) = 1499
## denominator =  539.4972
## tau = -0.143, 2-sided pvalue =0.046724
```

```
MonthlyMannKendall1 <- trend::smk.test(GaringerOzone.monthly.ts)
MonthlyMannKendall1
```

```
##
##  Seasonal Mann-Kendall trend test (Hirsch-Slack test)
##
## data:  GaringerOzone.monthly.ts
```

```
## z = -1.963, p-value = 0.04965
## alternative hypothesis: true S is not equal to 0
## sample estimates:
##    S varS
##  -77 1499
```

```
summary(MonthlyMannKendall1)
```

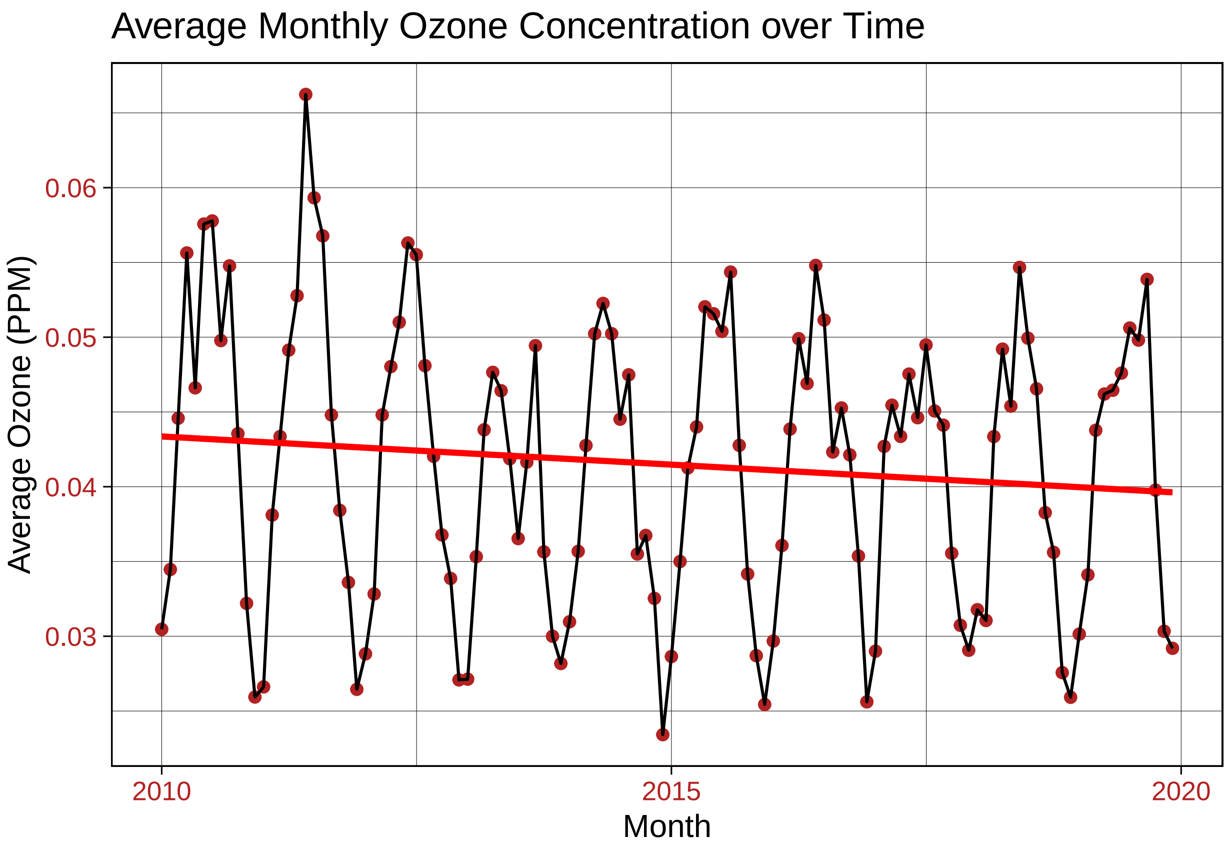
```
##
##  Seasonal Mann-Kendall trend test (Hirsch-Slack test)
##
## data: GaringerOzone.monthly.ts
## alternative hypothesis: two.sided
##
## Statistics for individual seasons
##
## H0
##                      S varS    tau      z Pr(>|z|)
## Season 1:   S = 0   15  125  0.333  1.252  0.21050
## Season 2:   S = 0   -1  125 -0.022  0.000  1.00000
## Season 3:   S = 0   -4  124 -0.090 -0.269  0.78762
## Season 4:   S = 0  -17  125 -0.378 -1.431  0.15241
## Season 5:   S = 0  -15  125 -0.333 -1.252  0.21050
## Season 6:   S = 0  -17  125 -0.378 -1.431  0.15241
## Season 7:   S = 0  -11  125 -0.244 -0.894  0.37109
## Season 8:   S = 0   -7  125 -0.156 -0.537  0.59151
## Season 9:   S = 0   -5  125 -0.111 -0.358  0.72051
## Season 10:   S = 0 -13  125 -0.289 -1.073  0.28313
## Season 11:   S = 0 -13  125 -0.289 -1.073  0.28313
## Season 12:   S = 0  11  125  0.244  0.894  0.37109
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Answer: A seasonal Mann-Kendall test is the most appropriate because it is the only method of montonic trend analysis that can be used with data that has a seasonal component.

13. Create a plot depicting mean monthly ozone concentrations over time, with both a geom_point and a geom_line layer. Edit your axis labels accordingly.

```
# 13
MonthlyOzonePlot <-
  ggplot(GaringerOzone.monthly,
         aes(x=Date,
             y=Monthly_AveragePPM)) +
  geom_point(color="firebrick") +
  geom_line() +
  xlab("Month") +
  ylab("Average Ozone (PPM)") +
  geom_smooth(method = lm, se=FALSE, color="red") +
  labs(title="Average Monthly Ozone Concentration over Time")
print(MonthlyOzonePlot)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



14. To accompany your graph, summarize your results in context of the research question. Include output from the statistical test in parentheses at the end of your sentence. Feel free to use multiple sentences in your interpretation. # Have ozone concentrations changed over the 2010s at this station? > Answer: Due to the p-value of less than 0.05, we can reject the null hypothesis and say that there is seasonality to the data, and that the data is not stationary. This means that the ozone concentrations did change over the 2010s at this station.

(MonthlyMannKendall tau = -0.143, 2-sided pvalue =0.046724 > summary(MonthlyMannKendall) Score = -77 , Var(Score) = 1499 denominator = 539.4972 tau = -0.143, 2-sided pvalue =0.046724 > > MonthlyMannKendall1 <- trend::smk.test(GaringerOzone.monthly.ts) > MonthlyMannKendall1

`Seasonal Mann-Kendall trend test (Hirsch-Slack test)`

data: GaringerOzone.monthly.ts z = -1.963, p-value = 0.04965 alternative hypothesis: true S is not equal to 0 sample estimates: S varS -77 1499

summary(MonthlyMannKendall1)

`Seasonal Mann-Kendall trend test (Hirsch-Slack test)`

data: GaringerOzone.monthly.ts alternative hypothesis: two.sided

Statistics for individual seasons

H0 S varS tau z Pr(>|z|)
Season 1: S = 0 15 125 0.333 1.252 0.21050

Season 2: S = 0 -1 125 -0.022 0.000 1.00000
Season 3: S = 0 -4 124 -0.090 -0.269 0.78762
Season 4: S = 0 -17 125 -0.378 -1.431 0.15241
Season 5: S = 0 -15 125 -0.333 -1.252 0.21050
Season 6: S = 0 -17 125 -0.378 -1.431 0.15241
Season 7: S = 0 -11 125 -0.244 -0.894 0.37109
Season 8: S = 0 -7 125 -0.156 -0.537 0.59151
Season 9: S = 0 -5 125 -0.111 -0.358 0.72051
Season 10: S = 0 -13 125 -0.289 -1.073 0.28313
Season 11: S = 0 -13 125 -0.289 -1.073 0.28313
Season 12: S = 0 11 125 0.244 0.894 0.37109
— Signif. codes: 0 '*' ***0.001*** *'' 0.01* '' 0.05 '.' 0.1 ' ' 1)

---

15. Subtract the seasonal component from the `GaringerOzone.monthly.ts`. Hint: Look at how we extracted the series components for the EnoDischarge on the lesson Rmd file.

16. Run the Mann Kendall test on the non-seasonal Ozone monthly series. Compare the results with the ones obtained with the Seasonal Mann Kendall on the complete series.

```
#15
GaringerOzone.monthly_Components <- as.data.frame(GaringerOzone.monthly.decomposed$time.series [1:120,]
GaringerOzone.monthly_Components <- mutate(GaringerOzone.monthly_Components,
                                           Average_PPM = GaringerOzone.monthly$Monthly_AveragePPM,
                                           Date = GaringerOzone.monthly$Date)

GaringerOzone_Nonseasonal <- GaringerOzone.monthly.ts - GaringerOzone.monthly.decomposed$time.series[,1

GaringerOzone.monthly_Components <- mutate(GaringerOzone.monthly_Components,
                                           Nonseasonal = GaringerOzone_Nonseasonal)

#16
MonthlyNSMannKendall <- Kendall::SeasonalMannKendall(GaringerOzone_Nonseasonal)

MonthlyNSMannKendall
```

```
## tau = -0.143, 2-sided pvalue =0.046724
```

```
summary(MonthlyNSMannKendall)
```

```
## Score =  -77 , Var(Score) = 1499
## denominator =  539.4972
## tau = -0.143, 2-sided pvalue =0.046724
```

```
MonthlyNSMannKendall1 <- trend::smk.test(GaringerOzone_Nonseasonal)
MonthlyNSMannKendall1
```

```
##
##  Seasonal Mann-Kendall trend test (Hirsch-Slack test)
##
```

```
## data:  GaringerOzone_Nonseasonal
## z = -1.963, p-value = 0.04965
## alternative hypothesis: true S is not equal to 0
## sample estimates:
##    S varS
##  -77 1499
```

```r
summary(MonthlyNSMannKendall1)
```

```
##
##  Seasonal Mann-Kendall trend test (Hirsch-Slack test)
##
## data: GaringerOzone_Nonseasonal
## alternative hypothesis: two.sided
##
## Statistics for individual seasons
##
## H0
##                       S varS    tau      z Pr(>|z|)
## Season 1:   S = 0   15  125  0.333  1.252  0.21050
## Season 2:   S = 0   -1  125 -0.022  0.000  1.00000
## Season 3:   S = 0   -4  124 -0.090 -0.269  0.78762
## Season 4:   S = 0  -17  125 -0.378 -1.431  0.15241
## Season 5:   S = 0  -15  125 -0.333 -1.252  0.21050
## Season 6:   S = 0  -17  125 -0.378 -1.431  0.15241
## Season 7:   S = 0  -11  125 -0.244 -0.894  0.37109
## Season 8:   S = 0   -7  125 -0.156 -0.537  0.59151
## Season 9:   S = 0   -5  125 -0.111 -0.358  0.72051
## Season 10:   S = 0 -13  125 -0.289 -1.073  0.28313
## Season 11:   S = 0 -13  125 -0.289 -1.073  0.28313
## Season 12:   S = 0  11  125  0.244  0.894  0.37109
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Answer: Comparing the results of this Seasonal Mann Kendall, it is revealed that they are the same even when subtracting seasonality. This would indicate that the seasonal component was not as strong as thought, or that the data is actually stationary over time.